9

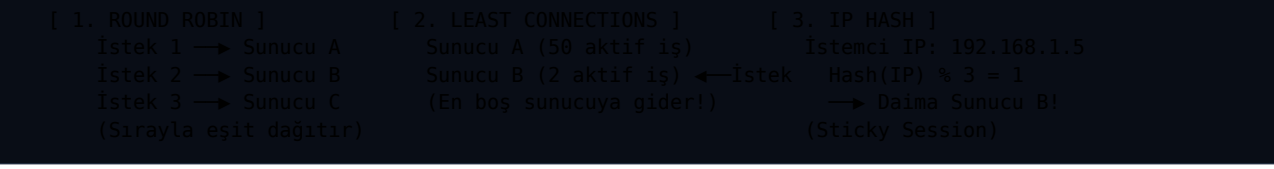
BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

Yük Dengeleme Algoritmaları: Round Robin, Least Connections, Weighted ve IP Hash

Load Balancing Algorithms: Round Robin, Least Conn, Weighted & IP Hash

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir bankada 3 gişe memuru olduğunu düşünün. Round Robin, sıra numarasını sırayla 1-2-3-1-2-3 diye memurlara dağıtan robot gibidir; ancak 1. memurun önündeki müşteri 1 saat sürecek devasa bir miras işlemi yapıyorsa o gişenin önü kilitlenir! Least Connections ise akıllı bir şeftir: Kafasını kaldırır, 'Hanginizin önünde daha az bekleyen insan var?' diye bakar ve yeni müşteriyi en boş memura gönderir. IP Hash ise 'Ali Bey sen daima 2 numaralı memura git' diyen kuraldır (aynı kullanıcı hep aynı sunucuya gider)!

🇬🇧 ENGLISH (ELI5)

Picture 3 bank tellers. Round Robin is a naive ticket dispenser handing turns sequentially: 1, 2, 3, 1, 2, 3. But if Teller 1 is stuck in a 2-hour corporate audit, their desk clogs completely while Teller 2 sits bored! Least Connections is a vigilant floor manager who checks: 'Who currently has the fewest active clients?' and dispatches the newcomer there. IP Hash calculates a hash of the customer's ID, consistently assigning that specific person to the exact same teller every visit (Session Stickiness)!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Yük dengeleme algoritmaları ikiye ayrılır: 1) Statik Algoritmalar: Sunucu yükünü umursamaz. Round Robin (sırayla), Weighted Round Robin (güçlü sunucuya daha fazla ağırlık, ör. 8 core makineye 4x istek), IP Hash (istemci IP'sine göre yapışkan oturum - Sticky Session). 2) Dinamik Algoritmalar: Gerçek zamanlı sunucu metriğini izler. Least Connections (aktif TCP bağlantısı en az olan sunucuya gönderir - uzun süren işlemler ve WebSockets için idealdir), Weighted Least Connections, Least Response Time (en hızlı yanıt veren sunucuyu tercih eder).

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Balancing algorithms categorize into: 1) Static Schemes: Agnostic to real-time server load. Round Robin (simple cycle), Weighted Round Robin (proportional weights assigned to heftier bare-metal nodes), IP/Consistent Hashing (binds client IP to persistent backend instances for stateful caching). 2) Dynamic Schemes: Real-time telemetry feedback. Least Connections (routes to node with minimum active sockets —vital for persistent WebSockets and heavy database queries), Weighted Least Connections, and Least Response Time (prioritizes instances demonstrating lowest TTFB).

⚡ Modern Java 25 Uygulaması (Loom & Records) / ⚡ Java 25 Implementation

```
// Java 25: Thread-Safe Weighted Round Robin & Least Connections Simülatörü
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class LoadBalancingSim {
    public record BackendNode(String id, int weight, AtomicInteger activeConnections) {}

    // 1. Round Robin
    public static class RoundRobinBalancer {
        private final List<BackendNode> nodes;
        private final AtomicInteger counter = new AtomicInteger(0);

        public RoundRobinBalancer(List<BackendNode> nodes) { this.nodes = nodes; }

        public BackendNode getNextNode() {
            int idx = Math.abs(counter.getAndIncrement() % nodes.size());
            return nodes.get(idx);
        }
    }

    // 2. Least Connections
    public static BackendNode selectLeastConnection(List<BackendNode> nodes) {
        return nodes.stream()
                .min((a, b) -> Integer.compare(a.activeConnections().get(),
b.activeConnections().get()))
                .orElseThrow();
    }
}

void main() {
    var n1 = new LoadBalancingSim.BackendNode("Node-A", 1, new AtomicInteger(42));
    var n2 = new LoadBalancingSim.BackendNode("Node-B", 1, new AtomicInteger(5));
    var n3 = new LoadBalancingSim.BackendNode("Node-C", 1, new AtomicInteger(18));

    var best = LoadBalancingSim.selectLeastConnection(List.of(n1, n2, n3));
    IO.println("Least Connection Seçimi: " + best.id() + " (Aktif: " + best.activeConnections() +
")");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı / ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

IP Hash ile oturum sabitleme (Sticky Session) dağıtık mimarilerde bir anti-pattern'dir! Bir sunucu çöktüğünde o sunucuya bağlı tüm kullanıcıların oturumu uçar ve yük dengesizleşir. Modern MAANG mimarilerinde sunucular tamamen durumsuz (Stateless) tutulur, oturumlar Redis'te saklanır ve Least Connections kullanılır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Sticky Sessions via IP Hash represent an architectural anti-pattern in cloud microservices. If an instance restarts, connected users lose state. Keep backends strictly stateless, store tokens in Redis, and leverage Least Connections.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Kısa süren tipik REST istekleri için Weighted Round Robin idealdir.

• Uzun süren veritabanı veya WebSocket bağlantıları için Least Connections şarttır.

• Durumsuz (stateless) tasarım sayesinde sunucu çökmeleri istemciyi etkilemez.

🎯 Key Takeaways for the Interview

• Weighted Round Robin excels for short-lived, uniform HTTP REST transactions.

• Least Connections is critical for long-lived WebSockets and compute-heavy transactions.

• Stateless servers combined with external token stores render sticky sessions obsolete.